Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Dictionaries, Tuples, and SetsIntroduction to computer progran

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS

Collections

In Python, **collections** are container data types.

- Put **more than one value** in a data structure and carry all the values around in one convenient package
- We have a bunch of values in **a single "variable"**
- We have **more than one place** "inside" the variable
- We also have **ways of finding** the different places in the variable

Python has 4 built-in collection data types:

- lists
- dictionaries
- tuples
- sets

A Story of Two Collections

Collection	Features	Visualization
List	a **linear ordered** collection of values list index their entities based on the **position** in the list.	
Dictionary	a **"bag of values"**, each with its own label the order of items in a dictionary is unpredictable dictionaries are indexed by keys (lookup tags)	

Dictionaries

- In Python, a dictionary is a mapping between a set of indices (*keys*) and a set of *values*
    - Items in a dictionary are **key-value pairs**
- Dictionaries are amongst the most powerful data collectors
- Dictionaries are also known as *hash tables* or *associative arrays*
    - They use a hash function for super-fast indexing of the keys
- Keys can't be any Python data type
    - Because keys are used for **indexing**, they should be **immutable**
    - Mutable objects do not support hashing
    - Common data types include strings and numbers
- Values can be any Python data type
    - Values can be **mutable** or **immutable**

Creating a Dictionary

There are two ways for creating a dictionary:

1. **Step-by-step**

```
d1 = dict()          # or d1 = {}, i.e., empty dictionary
d1['money'] = 12     # add a new key-value pair
d1['candy'] = 3      # add a new key-value pair
d1['tissues'] = 75   # add a new key-value pair
print(d1)
```

```
{'money': 12, 'candy': 3, 'tissues': 75}
```

2. **All-at-once**

```
d2 = {'money': 12, 'candy': 3, 'tissues': 75}
print(d2)
```

```
{'money': 12, 'candy': 3, 'tissues': 75}
```

Indexing and Manipulating a Dictionary

- **Dictionary Query**

```
d = {'money': 12, 'candy': 3, 'tissues': 75}
print(d['money'])
```

```
12
```

- **Update the Dictionary**

```
d['candy'] = d['candy'] + 1
print(d)
```

```
{'money': 12, 'candy': 4, 'tissues': 75}
```

- **Add New Items to the Dictionary**

```
d['pen'] = 2
print(d)
```

```
{'money': 12, 'candy': 4, 'tissues': 75, 'pen': 2}
```

- **Delete Items from the Dictionary**

```
del(d['money']) # del works on lists too...
print(d)
```

```
{'candy': 4, 'tissues': 75, 'pen': 2}
```

WARNING:

If the index is not a key in the dictionary, Python raises a dedicated `KeyError` error.

For instance:

```
d = {'money': 12, 'candy': 3, 'tissues': 75}
print(d['hello'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[7], line 2
      1 d = {'money': 12, 'candy': 3, 'tissues': 75}
----> 2 print(d['hello'])

KeyError: 'hello'
```

WARNING:

The `in` operator behaves differently with respect to lists and strings:

- For offset-indexed sequences (e.g., strings and lists), `x in y` checks whether `x` is an item in the sequence `y`
- For dictionaries, `x in y` checks whether `x` *is a key* of the dictionary `y`

```
L = [4, 6, 'hello', 7] # a list
print(7 in L)
s = 'coding with python' # a string
print('p' in s)
d = {'money': 12, 'candy': 3, 'tissues': 75} # a dict
print('hello' in d)
print('money' in d)
```

```
True
True
False
True
```

Dict Methods

`dict.keys()`

`dict.keys()` returns a *view object** containing all the keys of a dictionary.

You can conveniently cast into a list, a more common and flexible data object.

For instance:

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
k = dishes.keys() # a view object (list-like)
print(k)
k = list(k) # a proper list
print(k)
```

```
dict_keys(['eggs', 'sausage', 'bacon', 'spam'])
['eggs', 'sausage', 'bacon', 'spam']
```

WARNING: Dictionaries keep their items in the same order they were introduced**: `keys`, `values` and `items` (more on the latter later) are ordered according to this rule.

* View objects are dynamic overview on the dictionary's entries

** In Python 3.6 and earlier, dictionaries are fully unordered.

`dict.items()` and `dict.values()`

`dict.values()` returns a *view object* containing all the values of a dictionary.

`dict.items()` returns a *view object* containing all the key-value pairs of a dictionary.

You can conveniently cast into a list, a more common and flexible data object.

For instance:

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.values() # a view object (list-like)
v = list(v) # a proper list
print(v)
i = dishes.items() # a view object (list-like)
i = list(i) # a list of key-value tuples
print(i)
```

```
[2, 1, 1, 500]
[('eggs', 2), ('sausage', 1), ('bacon', 1), ('spam', 500)]
```

`dict.get()`

`dict.get(key, [x])` returns the value for `key`. If `key` is not found, returns `x`. If `x` is not given, it defaults to `None`.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.get('eggs', 10) # 'eggs' exists in dishes
print(v)
v = dishes.get('water', 10) # 'water' does not exist in dishes
print(v)
```

```
2
10
```

NOTE: Since `dict.get()` relies on a default value if key is not found, it will never raise a `KeyError`.

Pattern of usage: check whether a key is already in a dictionary and assuming a default value if the key is not there.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
# 'eggs' exists in dishes, so we take its value and add + 1
dishes['eggs'] = dishes.get('eggs', 0) + 1
print(dishes)
# 'water' does not exists in dishes, so we plug the default value 0 and add + 1
dishes['water'] = dishes.get('water', 0) + 1
print(dishes)
```

```
{'eggs': 3, 'sausage': 1, 'bacon': 1, 'spam': 500}
{'eggs': 3, 'sausage': 1, 'bacon': 1, 'spam': 500, 'water': 1}
```

`dict.get()` explained:

Let `d` be a dictionary. Saying `v = d.get(k, x)` is a shortcut for:

```
if k in d:
    v = d[k]
else:
    v = x
```

Similarly, saying `d[k] = d.get(k, x) + y` is a shortcut for:

```
if k in d:
    d[k] += y
else:
    d[k] = x + y
```

`dict.clear()`

`dict.clear()` deletes all items from a dictionary.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
dishes.clear()
print(dishes)
```

```
{}
```

NOTE: There is a strong analogy with lists:
`clear()` also works on lists (same effect ⟶ empty list).

`dict.pop()`

`dict.pop(key)` deletes the dictionary entry corresponding to the key `key` and returns its value.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.pop('sausage')
print(dishes)
print(v)        # value of the removed element
```

```
{'eggs': 2, 'bacon': 1, 'spam': 500}
1
```

NOTE: There is a strong analogy with lists:
`pop()` also returns the value when called on a list.

`dict.update()`

`dict.update(other_dict)` updates the dictionary `dict` with the key-value pairs from `other_dict`, overwriting existing keys.

```
d1 = {1: 10, 2: 20}
d2 = {3: 30, 4: 40}
d1.update(d2)
print(d1)
```

WARNING: If a key exists both `dict` and `other_dict`, the key-value pair in `other_dict` overwrites the one in `dict`.

```
d1 = {1: 10, 2: 20}
d2 = {3: 30, 2: 40} # we have an overlapping key (i.e., 2)
d1.update(d2)
print(d1) # the value associated to key 2 is taken from d2
```

```
{1: 10, 2: 40, 3: 30}
```

Iterating over Dicts

You can iterate over a dictionary with a very simple `for` loop:

- over its keys (way common)
- over its values (less common)

HINT: You can generate the sequence of keys and values with `dict.keys()` and `dict.values()`, respectively.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k in dishes.keys():
    print("Key is:", k, "and Value is:", dishes[k])
```

```
Key is: eggs and Value is: 2
Key is: sausage and Value is: 1
Key is: bacon and Value is: 1
Key is: spam and Value is: 500
```

HINT: If you iterate over keys, it's trivial to have both key and value. If you iterate over values, it's not trivial to get the corresponding key.

You can iterate **simultaneously** over keys and values of a dictionary with a very simple `for` loop.

HINT: You can get the key-value pairs with `dict.items()`.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k, v in dishes.items():
    print("Key is:", k, "and Value is:", v)
```

```
Key is: eggs and Value is: 2
Key is: sausage and Value is: 1
Key is: bacon and Value is: 1
Key is: spam and Value is: 500
```

What's going on here?

- there are **two iteration variables**, namely `k` and `v`
- `dict.items()` generates a list of key-value pairs (tuple)
- the first loop variable, namely `k`, contains the key in the key-value pair
- the second loop variable, namely `v`, contains the corresponding value in the key-value pair

Tuples

Tuples, like lists, are **ordered collections** of items.

Creating Tuples

You can create a tuple by enclosing items in round brackets:

```
t1 = (90, 98, 95) # homogeneous tuple
t2 = (90, 'hello', 95, [1, 2]) # heterogeneous tuple
t3 = () # empty tuple
```

or, you can omit the parenthesis altogether (this is called *tuple packing*):

```
t1 = 90, 98, 95 # homogeneous tuple
t2 = 90, 'hello', 95, [1, 2] # heterogeneous tuple
```

WARNNING: When you create a singleton tuple, you must include a tailing comma!

```
t = (90) # type(t) yields 'int'
print(type(t))
t = (90,) # now we have a 1-element tuple
print(type(t))
```

```
<class 'int'>
<class 'tuple'>
```

Unpacking

Tuples (and lists) can also be *unpacked**:

```
t = (90, 0, 'hello')
a, b, c = t # unpacking syntax
print(a, b, c)
```

```
90 0 hello
```

which is basically the same as doing:

```
a = t[0]
b = t[1]
c = t[2]
print(a, b, c)
```

```
90 0 hello
```

*: Unpacking is when you assign values from a tuple/list to a sequence of variables.

Common operations

Tuples support all of the common operations we have for lists, notably:

- indexing:
  `t[1]` yields `7` when `t=(98, 7, 'hello')`
- slicing:
  `t[1:3]` yields `(7, 'hello')` when `t=(98, 7, 'hello')`
- the `+` operator for concatenation:
  `(98, 7, 'hello')+(1,2,3)` yields `(98, 7, 'hello')+(1,2,3)`
- the `*` operator for replication:
  `(98, 7, 'hello')*3` yields `(98, 7, 'hello')*3`
- the `in` (and `not in`) operator for membership checking:
  `7 in (98, 7, 'hello')` yields `True`
- the `len()` function to calculate the length of a tuple:
  `len((98, 7, 'hello'))` yields `3`

Iterating over Tuples

You can also iterate over tuples, e.g. via `for` loops:

```
t = (98, 7, 'hello')
for item in t:
    print(item)
```

```
98
7
hello
```

Or, via a `while` loop by index:

```
t = (98, 7, 'hello')
i = 0
while i < len(t):
    print(t[i])
    i += 1
```

```
98
7
hello
```

Immutability

Yet, **tuples are immutable**, whereas **lists are mutable**.

```
t = (98, 7, 'hello')
t[-1] = 'python'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 2
      1 t = (98, 7, 'hello')
----> 2 t[-1] = 'python'

TypeError: 'tuple' object does not support item assignment
```

So you cannot change a tuple, just like you cannot change a string.

Workaround tuple immutability

Workarounds:

- create a new tuple that contains the manipulation of the original tuple
- cast tuple to list, change the list, cast list to tuple

As the latter is concerned:

```
t = (98, 7, 'hello')    # t is a tuple
print(t)
L = list(t) # convert tuple to list
print(L)
L.append(1) # do some manipulations
print(L)
t = tuple(L) # convert list to tuple
print(t)
```

```
(98, 7, 'hello')
[98, 7, 'hello']
[98, 7, 'hello', 1]
(98, 7, 'hello', 1)
```

Sets

A set is an **unordered** (unindexed) collection of items with **no** repetitions.

Creating sets

You can create an empty set thanks to the `set()` function.

And you can create a non-empty set thanks to the `set()` function or curly brackets.

For example:

```
# empty set
s1 = set()
# non-empty set with curly brackets
s2 = {'a', 1}
# non-empty set with set()
s3 = set(['a', 1])
```

WARNING: To not confuse the curly brackets to create dicts with the curly brackets to create sets.

Data types storable in a set

A set can contain only ***hashable*** items, notably:

- numbers
- strings
- tuples only if they do not contain mutable objects

and in no way they can contain mutable objects, notably:

- dictionaries
- lists
- tuples that contain mutable objects

In Python, the majority of immutable objects are also hashable.

Hashable

Hashing is a concept in computer science which is used to create high performance, pseudo random access data structures where large amount of data is to be stored and accessed quickly.

The hash of some data is deterministic by design and will never change if the data does not change

- this is why mutable objects are not hashable...

Also, matching the hashes of the data is usually faster than matching the data itself.

Sets and dictionary keys use hashing for fast access and look-up.

EXAMPLE: `if x in X`

- If `X` is a list, then Python scans the entire list until `x` is found: $O(n)$
- If `X` is a dictionary or set, `hash(x)` gives the position in memory: $O(1)$.

Common operations

- membership: `x in S`
- the `len(s)` function to calculate the cardinality (size) of a set `s`
- union (new set with elements in either `s1` or `s2`): `s1 | s2` or `s1.union(s2)`
- intersection (new set with elements in both `s1` and `s2`): `s1 & s2` or `s1.intersection(s2)`
- difference (new set with elements in `s1` but not in `s2`): `s1 - s2` or `s1.difference(s2)`
- symmetric difference (new set with elements in `s1` or `s2`, but not both): `s1 ^ s2` or `s1.symmetric_difference(s2)`

NOTE: Set operations yield a brand new set.

Set methods

- `s.add(x)` : adds `x` to the set `s` (if `x` already in `s`, nothing happens)
- `s.remove(x)` : removes `x` from the set `s` (if `x` is not in `s`, a `KeyError` is raised)
- `s.discard(x)` : removes `x` from the set `s` (if `x` is not in `s`, nothing happens)
- `s.update(x)` : adds elements from `x` into the set `s` (`x` can be any iterable, e.g., list, tuple or another set)
- `s.clear()` : removes all elements from `s`
- `s.pop()` : removes (and returns) an arbitrary item from `s`

NOTE: Set methods change the set in-place.

Recap on Collections

- `list` and `tuple` keep order; `set` and `dict` do not
- `dict` associates a key with a value; `set`, `tuple` and `list` contain just values
- `set` and `dict` keys only require items to be hashable; `list`, `tuples` and `dict` values do not
- `set` and `dict` keys forbid duplicates; `list`, `tuples` and `dict` values allow duplicates.

Checking for membership inside a `set` or `dict` is super-fast thanks to hashing.

`tuple` and `list` require scanning the list/tuple until the element is found.

Exercise session

Find the exercises for the exam session: here